ML_Lab2_Datapreprocessing

August 31, 2026

1 Outliers detection and handling

1.1 Load the CSV Files

[ ]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

[ ]:
```
df = pd.read_csv("CIE1.csv")
```

[ ]:
```
df
```

[ ]:
```
df.shape
```

1.2 Boxplot

A Box Plot, also known as a Box-and-Whisker Plot, is a graphical representation of the distribution of a dataset, showing its minimum, first quartile (Q1), median, third quartile (Q3), and maximum. It is particularly useful for identifying outliers and understanding the spread and symmetry of the data.

[ ]:
```
sns.boxplot(df.Marks)
```

[ ]:
```
plt.hist(df.Marks, bins=20, rwidth=0.8)
plt.xlabel('CIE-1 marks')
plt.ylabel('Count')
plt.show()
```

[ ]:
```
df.Marks.describe()
```

[ ]:
```
df.Marks.mean(), df.Marks.std()
```

1.2.1 3-sigma rule.

Calculates the upper bound (UB) and lower bound (LB) for values in the column Marks of a DataFrame df using the mean and standard deviation of the column. This is commonly used in statistical analysis to identify outliers based on the 3-sigma rule.

[ ]:
```
UB = df.Marks.mean() + 3*df.Marks.std()
LB = df.Marks.mean() - 3*df.Marks.std()
```

[ ]:
```
UB, LB
```

[ ]:
```
df[(df.Marks>UB) | (df.Marks<LB)]
```

[ ]:
```
df_new_std_dev = df[(df.Marks<UB) & (df.Marks>LB)]
df_new_std_dev.shape
```

1.3 Formula

The formula to calculate the Z-Score is: $=(-)/$

[ ]:
```
df['zscore'] = ( df.Marks - df.Marks.mean() ) / df.Marks.std()
df
```

[ ]:
```
df[df['zscore']>3]
```

[ ]:
```
df_new_zscore = df[df.zscore<3]
df_new_zscore.shape
```

The interquartile range (IQR), which is the difference between Q3 and Q1, is used to identify potential outliers in the dataset.

Interquartile Range (IQR) Calculation: $= 3 - 1$

Outlier Detection Using IQR: Outliers are often defined as values that lie outside the following range:

Lower Bound $= 1 - 1.5\times$ Upper Bound $= 3 + 1.5\times$ Values outside this range are considered outliers.

[ ]:
```
Q1 = df.Marks.quantile(0.25)
Q3 = df.Marks.quantile(0.75)
Q1, Q3
```

[ ]:
```
df.Marks.std()
```

[ ]:
```
IQR = Q3 - Q1
IQR
```

[ ]:
```
LB = Q1 - 1.5*IQR
UB = Q3 + 1.5*IQR
UB, LB
```

[ ]:
```
df
```

[ ]:
```
df_new_IQR = df[(df.Marks>LB)&(df.Marks<UB)]
df_new_IQR.shape
```

```
[ ]: df1 = df
     df1
```

```
[ ]: import numpy as np
     df1['Marks'] = np.where(df1['Marks'] > UB, UB, df1['Marks'])  #quantile based
      ↪flooring
     df1['Marks'] = np.where(df1['Marks'] < LB, LB, df1['Marks'])  #quantile based
      ↪capping
     df1.shape
```

```
[ ]: df1.describe()
```

2 Encoding Categorical Variables

```
[ ]: from sklearn.preprocessing import LabelEncoder
     df_labelencoding = df1

     df_labelencoding
```

```
[ ]: label_encoder = LabelEncoder()
     df_labelencoding['Passing_in_CIE']=label_encoder.
      ↪fit_transform(df_labelencoding['Passing_in_CIE'])
```

```
[ ]: df_labelencoding
```

```
[ ]: from sklearn.preprocessing import OneHotEncoder
     onehot_encoder = OneHotEncoder(sparse_output=False)
     encoded_data = onehot_encoder.fit_transform(df[['Passing_in_CIE']])
     df_encoded = pd.DataFrame(encoded_data, columns=onehot_encoder.
      ↪get_feature_names_out(['Passing_in_CIE']))

     df_V = df1.join(df_encoded)
     df_V
```

```
[ ]: df_V
```

2.1 Part 1: Outlier Detection and Handling

Task 1: Visual & Statistical Inspection Plot box plots and histograms for Base_Salary and Overtime_Hours.

Compute summary statistics (mean, std, Q1, median, Q3).

Task 2: 3-Sigma / Z-Score Filtering Calculate Z-score for Base_Salary and isolate records where $|Z| > 3$.

Filter these extreme values and inspect the change in mean and standard deviation.

Task 3: IQR Capping (Winsorizing) Compute IQR $= Q3 - Q1$ for Overtime_Hours.

Calculate bounds: Lower $= Q1 - 1.5 \times$ IQR and Upper $= Q3 + 1.5 \times$ IQR.

Cap/clip the values using pd.Series.clip().

2.2 Part 2: Encoding Categorical Features into Numericals

Task 4: Binary & Ordinal Encoding Binary Encoding: Map Promoted_This_Year (No / Yes) into numerical values (0 / 1).

Ordinal Encoding: Map Job_Level into numerical ranks (Junior $\rightarrow$ 0, Mid-Level $\rightarrow$ 1, Senior $\rightarrow$ 2, Executive $\rightarrow$ 3) preserving logical hierarchy.

Task 5: One-Hot Encoding for Nominal Categorical Variables. Transform Department into binary indicator columns using pd.get_dummies() or OneHotEncoder. Set drop_first=True to eliminate multicollinearity (dummy variable trap).

Verify that all categorical string features have been completely converted to numerical types (int or float).

[ ]: